



UNIVERSIDAD DE MÁLAGA



Grado en Ingeniería Informática

Desarrollo de un sistema inteligente explicable basado en Graph-RAG y Análisis Formal de Conceptos

Development of an explainable intelligent system based on Graph-
RAG and Formal Concept Analysis

Realizado por
Miguel Ángel Caballero Serrano

Tutorizado por
Ángel Mora Bonilla

Departamento
Matemática Aplicada

MÁLAGA, junio 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA
GRADUADO EN INGENIERÍA INFORMÁTICA

**Desarrollo de un sistema inteligente explicable basado
en Graph-RAG y Análisis Formal de Conceptos**

**Development of an explainable intelligent system based on
Graph-RAG and Formal Concept Analysis**

Realizado por
Miguel Ángel Caballero Serrano

Tutorizado por
Ángel Mora Bonilla

Departamento
Matemática Aplicada

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2026

Fecha defensa: julio de 2025

Resumen

El presente Trabajo Fin de Grado se enmarca dentro del uso de los Modelos de Lenguaje de Gran Tamaño (LLMs) y los sistemas de recuperación de información mediante grafos (Graph-RAG). Actualmente, los modelos de lenguaje cometen errores (conocidos como alucinaciones) y no explican cómo llegan a sus conclusiones. Aunque el uso de Graph-RAG mejora la búsqueda de contexto, carece del rigor matemático necesario para garantizar que las respuestas sean totalmente ciertas.

El objetivo principal de este trabajo es diseñar una arquitectura que integre el Análisis Formal de Conceptos (FCA). De este modo, FCA actúa como una guía lógica y matemática para controlar cómo el modelo genera el texto.

Para el desarrollo del proyecto se ha implementado una aplicación interactiva en el lenguaje R utilizando Shiny. En ella, se usa el paquete fcaR para extraer el conocimiento de los datos en forma de retículos de conceptos y reglas de implicación lógica. Además, este sistema se conecta con un modelo de lenguaje que se ejecuta en el propio ordenador del usuario (en local) usando el paquete ellmer y la herramienta Ollama, lo que asegura la privacidad de los datos.

Este trabajo demuestra que inyectar reglas lógicas al modelo reduce de forma importante las alucinaciones frente a los sistemas convencionales. El usuario dispone de diagramas visuales jerárquicos que le permiten ver y entender los pasos lógicos que el sistema ha seguido para dar una respuesta. Así, se consigue un sistema de inteligencia artificial fiable y fácil de explicar.

Palabras clave: Análisis de Conceptos Formales, Graph-RAG, Modelos de Lenguaje de Gran Tamaño, Interfaz de Usuario, Inteligencia Artificial Fiable, Lenguaje de Programación R.

Abstract

This Final Year Dissertation is framed within the use of Large Language Models (LLMs) and graph-based retrieval-augmented generation systems (Graph-RAG). Currently, language models make errors (known as hallucinations) and do not explain how they reach their conclusions. Although the use of Graph-RAG improves context retrieval, it lacks the mathematical rigor necessary to guarantee that the answers are entirely accurate.

The main objective of this work is to design an architecture that integrates Formal Concept Analysis (FCA). In this way, FCA acts as a logical and mathematical guide to control how the model generates text.

For the development of the project, an interactive application has been implemented in the R programming language using Shiny. In it, the `fcaR` package is used to extract knowledge from the data in the form of concept lattices and logical implication rules. Furthermore, this system connects with a language model that runs on the user's own computer (locally) using the `ellmer` package and the Ollama tool, which ensures data privacy.

This work demonstrates that injecting logical rules into the model significantly reduces hallucinations compared to conventional systems. The user is provided with hierarchical visual diagrams that allow them to see and understand the logical steps the system has followed to provide an answer. Thus, a trustworthy and easy-to-explain artificial intelligence system is achieved.

Keywords: Formal Concept Analysis, Graph-RAG, Large Language Models, User Interface, Trustworthy Artificial Intelligence, R Programming Language.

Índice

Resumen.....	2
Abstract.....	3
Índice	5
Índice de Figuras.....	7
Introducción	9
1.1. Motivación.....	9
1.2. Objetivos.....	10
1.3. Estructura de la memoria.....	11
Metodología.....	13
2.1. Requisitos del sistema.....	13
2.1.1. Requisitos Funcionales.....	13
2.1.2. Requisitos No Funcionales.....	14
2.2. Diseño.....	14
2.2.1. Casos de uso.....	15
2.3. Implementación.....	16
2.4. Documentación.....	16
Tecnologías usadas.....	17
3.1. Entorno de Desarrollo y Programación.....	17
3.2. Arquitectura Base y Lógica Matemática.....	17
3.3. Inteligencia Artificial y Privacidad.....	18
3.4. Interfaz de Usuario y Explicabilidad Visual.....	18
3.5. Evaluación de Resultados.....	18
3.6. Herramientas de Gestión y Documentación.....	18
Marco teórico.....	21
4.1. RAG.....	21
4.1.1. Ciclo.....	22
4.1.2. Limitaciones en entornos prácticos.....	22
4.2. Graph-RAG.....	23
4.2.1. Categorías según las estructuras de grafos.....	23
4.2.2. Beneficios ante sistemas basados en bases de datos vectoriales.....	24
4.3. RAG Tradicional vs. Graph-RAG.....	24
4.4. FCA.....	25
4.4.1. Conceptos previos.....	25
4.4.2. Deficiones.....	25
4.4.3. Ejemplo sencillo con el paquete fcaR.....	26
FCA en Graph-RAG.....	29
5.1. Síntesis Neuro-Simbólica	29
5.2. LLM como analizador semántico y traductor formal.....	30

5.3.	FCA como autoridad lógica.....	30
5.4.	Explicabilidad y trazabilidad.....	31
5.5.	Caso ilustrativo.....	31
Desarrollo aplicación Shiny.....		33
Validación y pruebas.....		33
Conclusiones y líneas futuras.....		34
Referencias.....		34
Apéndice A.....		37
Manual de Usuario.....		37

Índice de Figuras

Figura 1 Diagrama UML.....	15
Figura 2 RAG Coste-Rendimiento.....	21
Figura 3 Comparación entre RAG Y Graph-RAG.....	24

1

Introducción

1.1. Motivación

En los últimos años, los Modelos de Lenguaje de Gran Tamaño (LLMs) se han convertido en herramientas muy populares gracias a su gran capacidad para procesar texto y responder preguntas de forma natural. Sin embargo, su funcionamiento interno se basa en la predicción probabilística, en lugar de en el razonamiento deductivo sobre hechos reales, es decir, intentan adivinar cuál es la siguiente palabra más adecuada en lugar de razonar sobre hechos reales. Esta característica provoca uno de sus mayores defectos: la invención de información o alucinaciones, junto con la incapacidad de explicarle al usuario cómo han llegado a esa conclusión. Siendo esto fundamental en sectores donde la precisión es vital, como la medicina, las finanzas o el derecho.

Para mitigar este problema, se desarrollaron los sistemas RAG (Generación Aumentada por Recuperación), los cuales buscan información en bases de conocimientos externas y se la entregan al modelo para que base sus respuestas en datos reales. Dado que el RAG tradicional a menudo perdía el contexto general, se evolucionó hacia el Graph-RAG, una técnica que organiza la información en forma de grafo (una red que conecta conceptos y sus relaciones) para mejorar la comprensión global del texto. No obstante, aunque el Graph-RAG mejora la calidad del contexto, sigue presentando una limitación: conecta información de forma semántica y asociativa, pero carece de un rigor matemático que garantice que las conexiones conformen una ruta de inferencia determinista y libre de contradicciones.

Para cubrir esta carencia, se recurre al Análisis Formal de Conceptos (FCA), una teoría matemática basada en la teoría de retículos y el orden. El FCA es capaz de analizar un conjunto de

datos, agrupar los objetos que comparten características comunes, (conceptos formales) y, lo más importante, extraer reglas fijas y absolutas entre ellos, conocidas como implicaciones lógicas.

La motivación central de este trabajo nace de la necesidad de integrar lo mejor de ambos enfoques. El objetivo es utilizar la estructura del Análisis Formal de Conceptos para que actúe como una autoridad lógica sobre el modelo de lenguaje. De este modo, en lugar de permitir que la IA asocie ideas libremente con el riesgo de equivocarse, se le imponen unas reglas matemáticas estrictas extraídas de los propios conjuntos de datos.

El resultado que persigue esta integración es transformar una herramienta probabilística y opaca en un sistema de IA Fiable y transparente. Al obligar al modelo a seguir las reglas del FCA, no solo se consigue reducir de forma drástica la posibilidad de generar alucinaciones, sino que se logra que el sistema sea totalmente explicable. Así, cualquier usuario podrá ver la ruta lógica exacta que el sistema ha seguido para resolver su consulta paso a paso.

1.2. Objetivos

El objetivo principal es el diseño y desarrollo de un sistema de IA explicable que integre la arquitectura Graph-RAG con FCA. El propósito fundamental es mitigar de forma controlada las alucinaciones de los LLMs y garantizar que todas las respuestas generadas cuenten con una trazabilidad lógica y matemática absoluta.

Para estructurar y alcanzar esta meta, el proyecto se desglosa en los siguientes cuatro objetivos específicos:

- **Investigación del estado del arte de las arquitecturas RAG y Graph-RAG:** Este objetivo inicial se centra en analizar la evolución de los sistemas de recuperación de información para entender sus capacidades y fallos actuales. Se estudiarán las limitaciones del RAG tradicional, el cual recupera fragmentos de texto basándose únicamente en la similitud estadística, lo que a menudo provoca la pérdida del contexto general y dificultades para realizar razonamientos complejos. Posteriormente, se investigará a fondo el paradigma Graph-RAG, que organiza el conocimiento externo en forma de grafos. Se analizará cómo esta estructura permite a la IA conectar información dispersa y mejorar la comprensión global del texto.
- **Investigación sobre la integración de FCA en los grafos de Graph-RAG:** Aunque Graph-RAG mejora la búsqueda de contexto, sus grafos conectan información de forma visual y

semántica, pero carecen de leyes matemáticas estrictas que aseguren que esas conexiones sean lógicamente ciertas. El objetivo es investigar cómo puede aportar el rigor matemático que le falta al sistema. Se analizará el proceso para transformar los datos del grafo en contextos formales que permitan extraer jerarquías exactas (retículos de conceptos) y reglas fijas (bases de implicaciones lógicas). Estas reglas actuarán como una autoridad que dictará al modelo de lenguaje qué es verdad y qué no, bloqueando posibles alucinaciones lógicas.

- **Desarrollo de una aplicación en Shiny como motor de inferencia neuro-simbólico:** Consistirá en construir una herramienta de software que conecte la base matemática con el modelo de lenguaje. Los hitos técnicos incluyen: la utilización del paquete matemático fcaR en el entorno de programación R para extraer automáticamente los retículos de conceptos y las reglas lógicas a partir de los datos; la integración con modelos de lenguaje ejecutados en local mediante el servidor Ollama y el paquete ellmer, asegurando la total privacidad de la información; y el desarrollo de la interfaz de usuario en Shiny, que incluirá herramientas visuales para mostrar de forma gráfica e interactiva el razonamiento exacto de la IA.
- **Validación de la arquitectura y análisis de resultados:** El objetivo final es demostrar mediante pruebas prácticas que la solución desarrollada es más segura y fiable que las alternativas actuales. Para ello, se realizará un análisis comparativo directo entre un sistema Graph-RAG convencional y la nueva arquitectura guiada por FCA. Se cuantificará la reducción de información inventada utilizando marcos de evaluación automatizada (como el framework RAGAS, enfocado en la fidelidad de la respuesta) para comprobar que el modelo respeta las restricciones matemáticas impuestas. Asimismo, se comprobará la capacidad del sistema para emitir una justificación auditable para cada recomendación, demostrando que se ha transformado un modelo probabilístico opaco en una IA Fiable y transparente.

1.3. Estructura de la memoria

El presente documento se divide en las siguientes secciones:

- **Resumen / Abstract:** Se realiza una síntesis breve e independiente que engloba el problema, los objetivos y los resultados obtenidos en el proyecto.
- **Introducción:** Se expone el contexto del proyecto, justificando la motivación para mitigar las alucinaciones de la IA, se definen los objetivos principales y específicos a alcanzar y, por último, se detallan las secciones de la memoria.

- **Metodología y tecnologías usadas:** Se describe el proceso de trabajo seguido y las herramientas como lenguajes y librerías utilizadas junto con los componentes adicionales necesarios.
- **Marco Teórico (Estado del Arte):** Se establecen los fundamentos teóricos necesarios, analizando RAG, Graph-RAG y la teoría del Análisis Formal de Conceptos.
- **FCA en Graph-RAG:** Se recoge cómo integrar el conocimiento de FCA en los grafos de Graph-RAG.
- **Desarrollo de la Aplicación Shiny:** Se presenta la implementación técnica de la interfaz visual con la que interactuará el usuario y cómo se muestran los diagramas lógicos para la explicabilidad.
- **Validación y pruebas:** Demostración final de que el sistema desarrollado realmente mitiga los errores frente a sistemas Graph-RAG convencionales.
- **Conclusiones y líneas futuras:** Se realiza un balance sobre si se han cumplido los objetivos y propone nuevas ideas o mejoras que se podrían investigar en el futuro.
- **Referencias:** Se recogen las diferentes referencias bibliográficas que se consultaron durante el desarrollo.
- **Apéndice:** Se detallan el material extra y el Manual de Usuario.

2

Metodología

Para el desarrollo de este trabajo se ha adoptado una metodología Agile, basada en el desarrollo iterativo e incremental. Esta metodología plantea la división del trabajo en fases o sprints, al término de cada una de las cuales se obtiene una parte funcional y evaluable del proyecto, lo que permite detectar y corregir errores de forma temprana, minimizando su propagación.

A diferencia del Modelo en Cascada, que sigue un enfoque lineal y secuencial en el que cada fase (requisitos, diseño, implementación, pruebas y despliegue) debe completarse antes de iniciar la siguiente, la metodología Agile ofrece mayor flexibilidad ante el cambio. Aplicar un enfoque en cascada en este trabajo supondría una limitación significativa, dado que tanto los requisitos como los obstáculos técnicos pueden evolucionar a lo largo del proceso, lo que obligaría a rediseñar el proyecto desde el inicio ante cualquier modificación.

2.1. Requisitos del sistema

Para garantizar el correcto funcionamiento de la arquitectura propuesta, se han establecido los siguientes requisitos funcionales y no funcionales.

2.1.1. Requisitos Funcionales

- **RF1. Carga y procesamiento de documentos:** El sistema debe permitir la subida de documentos, a partir de los cuales se extraerán los objetos, atributos y relaciones presentes en los textos.
- **RF2. Extracción de lógica matemática:** El sistema debe utilizar el paquete fcaR para analizar los datos extraídos y calcular automáticamente las agrupaciones y las reglas matemáticas fijas.

- **RF3. Interfaz de chat interactiva:** Se debe proporcionar un espacio interactivo donde se puedan realizar consultas sobre los documentos utilizando lenguaje natural.
- **RF4. Generación de respuestas guiadas:** Ante una consulta, el sistema debe buscar en su base matemática y enviar al modelo de lenguaje tanto el contexto como las reglas lógicas que el modelo debe obedecer.
- **RF5. Validación de errores:** Antes de mostrar la respuesta, el sistema debe comprobar lógicamente que el texto generado por la IA no contradice ninguna regla matemática del conjunto de datos.
- **RF6. Visualización del razonamiento:** Se deben generar gráficos interactivos mediante visNetwork para representar el camino lógico que ha seguido la IA para llegar a su conclusión.

2.1.2. Requisitos No Funcionales

- **RNF1. Privacidad total y seguridad:** El sistema debe funcionar de manera completamente local utilizando la herramienta Ollama, sin enviar datos a servidores externos.
- **RNF2. Fiabilidad y veracidad:** Se debe garantizar que las respuestas generadas sean ciertas desde un punto de vista matemático, minimizando la probabilidad de alucinaciones.
- **RNF3. Trazabilidad y auditoría:** Cada respuesta emitida por la IA debe ser transparente, permitiendo revisar los pasos exactos que tomó el sistema.
- **RNF4. Fluidez y tiempo de respuesta:** Se deben implementar procesos asíncronos en la interfaz Shiny para evitar que la aplicación se bloquee mientras el modelo de lenguaje procesa la respuesta.
- **RNF5. Usabilidad:** La aplicación debe presentar un diseño accesible, de modo que no se requieran conocimientos avanzados en programación o matemáticas para auditar las respuestas del modelo.

2.2. Diseño

Las herramientas que se han utilizado durante el transcurso del proyecto serán definidas a continuación, donde el lenguaje de programación principal sería R, dado que constituye el núcleo del ecosistema del paquete matemático fcaR y facilita la orquestación de modelos de lenguaje mediante la librería ellmer. Además, el entorno interactivo se desarrollaría utilizando el marco de trabajo Shiny,

pues proporciona la infraestructura necesaria para desplegar aplicaciones web directamente desde R.

En esta fase se definió, a nivel estético y funcional, cuáles serían los resultados esperados y la arquitectura visual del sistema. Para ello, se comenzó realizando varios bocetos o mockups de la aplicación utilizando herramientas de diseño vectorial colaborativo como Figma. A través de este proceso, se estructuró la interfaz en torno a los módulos críticos del sistema neuro-simbólico: un panel de ingesta de documentos, un entorno de chat conversacional y un área de explicabilidad visual. Esto se realizó para prever cuáles iban a ser las funcionalidades disponibles y cómo se iban a distribuir en la pantalla sin sobrecargar al usuario.

Además, otro paso fundamental en esta fase fue la elaboración de diagramas UML para visualizar la arquitectura del software.

2.2.1. Casos de uso

Un caso de uso constituye una representación gráfica de los requisitos funcionales, permitiendo observar cómo interactúa el usuario final con el sistema propuesto.

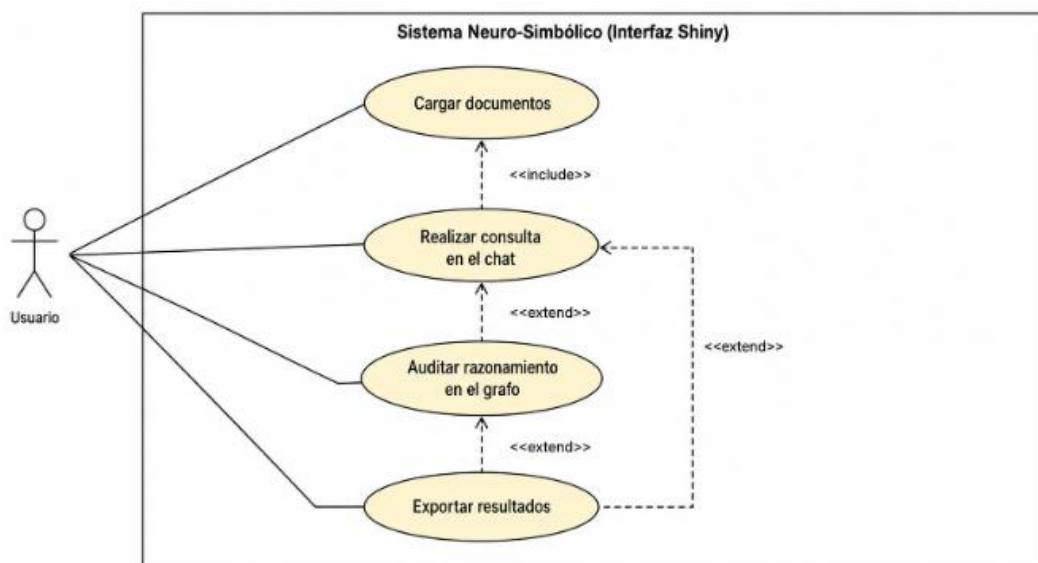


Figura 1 Diagrama UML

En el contexto de la arquitectura FCA-RAG desarrollada, el flujo de trabajo principal del usuario se articula en los siguientes pasos:

- **Carga y procesamiento de contexto:** Una vez proporcionados, el sistema procesa la información en segundo plano, extrayendo las entidades y delegando en fcaR el cálculo del retículo de conceptos y la base de implicaciones lógicas.

- **Interacción conversacional (Chat):** Una vez formalizado el contexto, el usuario puede introducir consultas en lenguaje natural, que desencadena el proceso de recuperación, donde el sistema inyecta las reglas matemáticas en el prompt y se comunica con el Modelo de Lenguaje para generar una respuesta.
- **Auditoría y Explicabilidad Visual:** Tras recibir la respuesta del asistente, el usuario puede navegar a una pestaña dedicada a la visualización. En ella, se renderiza de forma interactiva el Diagrama de Hasse, permitiendo al usuario ver la ruta que el sistema ha seguido para alcanzar su conclusión.
- **Exportación de resultados:** Por último, la aplicación dispone de funcionalidades de descarga en distintas partes de la interfaz. Dependiendo de la etapa en la que se encuentre, el usuario puede guardar el historial de la conversación o exportar la traza lógica y las reglas matemáticas calculadas para su posterior análisis sin necesidad de reprocesar los documentos desde cero.

2.3. Implementación

Para la elaboración del código, se han ido implementando de forma incremental las distintas funcionalidades disponibles. Se intercalaron fases de programación, de testeo y de revisión, teniendo en cuenta las funcionalidades dependientes de otras.

2.4. Documentación

A lo largo de la realización del proyecto, se ha ido documentando los cambios realizados de manera incremental.

3

Tecnologías usadas

El desarrollo se apoya en un conjunto de tecnologías de código abierto centradas principalmente en el ecosistema de programación R. La elección de estas herramientas persigue un doble objetivo: por un lado, permitir el procesamiento matemático complejo necesario para extraer la lógica de los datos; y por otro, garantizar que los Modelos de Lenguaje operen de forma local, interactiva y privada.

A continuación, se detallan las principales tecnologías empleadas en la arquitectura del sistema:

3.1. Entorno de Desarrollo y Programación

- **Lenguaje R:** Es el lenguaje de programación principal utilizado en el proyecto. Actúa como la columna vertebral que une el procesamiento de datos, la lógica matemática, la inteligencia artificial y la interfaz visual.
- **RStudio:** Es el entorno de desarrollo integrado (IDE) que se ha utilizado para escribir, organizar y probar todo el código escrito en el lenguaje R de una manera cómoda y eficiente.

3.2. Arquitectura Base y Lógica Matemática

- **Graph-RAG:** Es el paradigma o concepto de diseño principal del proyecto. A diferencia de los buscadores tradicionales, organiza la información en forma de grafo para que el sistema comprenda el contexto general de los datos.

- **fcaR:** Es un paquete matemático de R encargado del Análisis Formal de Conceptos. Toma la información del grafo y extrae automáticamente agrupaciones y reglas lógicas fijas.

3.3. Inteligencia Artificial y Privacidad

- **Ollama:** Es una aplicación que permite descargar y ejecutar LLMs directamente en el ordenador del usuario. Su uso garantiza que la información y los documentos procesados nunca viajen por internet, asegurando una privacidad total de los datos.
- **ellmer:** Es una librería de R que actúa como puente de comunicación. Permite que nuestro código en R envíe las preguntas del usuario y las reglas lógicas extraídas por fcaR al modelo de inteligencia artificial que está funcionando dentro de Ollama.

3.4. Interfaz de Usuario y Explicabilidad Visual

- **Shiny:** Es una herramienta de R que permite construir aplicaciones web interactivas sin necesidad de programar en lenguajes web tradicionales. Se ha utilizado para diseñar la pantalla principal donde el usuario sube sus documentos y chatea con el sistema.
- **visNetwork:** Es un paquete de R especializado en dibujar redes interactivas. Se utiliza en la aplicación para mostrarle al usuario el grafo y las rutas lógicas (Diagramas de Hasse) que el sistema ha seguido, para entender por qué la IA dio una respuesta específica.

3.5. Evaluación de Resultados

- **RAGAS:** Es un marco de evaluación diseñado para puntuar sistemas de Inteligencia Artificial. En este trabajo, se ha utilizado de forma secundaria para comprobar matemáticamente si las reglas inyectadas han reducido realmente las alucinaciones (evaluando la fidelidad y relevancia de las respuestas generadas).

3.6. Herramientas de Gestión y Documentación

- **GitHub:** Plataforma utilizada para alojar las distintas versiones del código fuente a lo largo del desarrollo. Ha permitido tener un control de cambios seguro, organizando el avance del software paso a paso.

- **Microsoft Word:** Procesador de textos utilizado para la elaboración de la memoria escrita de este Trabajo de Fin de Grado.

Marco teórico

4.1. RAG

Retrieval-Augmented Generation (Generación Aumentada por Recuperación) surge como solución a los altos costes computacionales y la complejidad técnica que supone entrenar o ajustar Modelos de Lenguaje de Gran Tamaño desde cero para dominios específicos. Aquí entra RAG, que es una técnica para mejorar la precisión y fiabilidad de los modelos generativos de IA con datos obtenidos de fuentes externas. En la siguiente gráfica podemos observar que se encuentra en un punto intermedio entre obtener un gran desempeño en cuanto a rendimiento y precisión sin que suponga gran coste de implementación.



Figura 2 Gráfica Coste-Rendimiento LLMs

4.1.1. Ciclo

El ciclo de funcionamiento de un sistema RAG tradicional se estructura en tres fases fundamentales:

1. Preparación e Indexación del Conocimiento Para que la información pueda ser consultada, primero debe ser agregada a una base de datos de la siguiente manera:

- 1. Fragmentación (Chunking):** Los documentos originales se dividen en fragmentos de texto más pequeños y manejables.
- 2. Vectorización (Embeddings):** Cada fragmento de texto se procesa mediante un modelo de representación para convertirlo en un embedding, representaciones vectoriales densas en un espacio matemático de alta dimensionalidad que capturan el significado semántico del texto.
- 3. Almacenamiento:** Estos vectores se almacenan en una Base de Datos Vectorial, sirviendo como índice de conocimiento.

2. Recuperación del Contexto (Retrieval) Cuando el usuario realiza una consulta o Query en lenguaje natural, en lugar de enviarla directamente al LLM para que responda de memoria, el sistema realiza una búsqueda de información, se transforma en un embedding; se realiza una búsqueda matemática para encontrar los fragmentos de documento en la base de datos cuyos vectores sean más similares al vector de la pregunta; y el sistema recupera los fragmentos de texto más relevantes.

3. Integración y Generación En la fase final, los documentos recuperados se empaquetan en un bloque denominado contexto, que se adjunta a la consulta original del usuario y se envía al LLM. El modelo genera una respuesta fundamentada estrictamente en los datos proporcionados.

En definitiva, este proceso garantiza que la información devuelta por el LLM esté basada en el conocimiento propio y actualizado del usuario, mitigando las alucinaciones y permitiendo referenciar exactamente qué documentos dieron origen a la respuesta.

4.1.2. Limitaciones en entornos prácticos.

- **Dificultad en la comprensión de consultas complejas dentro de dominios especializados,** ya que los métodos tradicionales de recuperación no logran abarcar el razonamiento de múltiples pasos ni captar los matices semánticos profundos.

- **La integración de conocimiento distribuido**, ya que la práctica necesaria de fragmentar los textos para su indexación sacrifica contexto esencial.
- **Las restricciones propias de los LLM**, concretamente la limitación de sus ventanas de contexto, obligan a truncar la información, destruyendo el flujo lógico y las dependencias a largo plazo.
- **Eficiencia y escalabilidad**, puesto que la búsqueda en bases de conocimiento a gran escala incrementa los costes computacionales y la latencia, observando además una degradación en la precisión del sistema conforme la base de datos se expande.

4.2. Graph-RAG

Para superar estas deficiencias, emerge el paradigma de la Generación Aumentada por Recuperación basada en Grafos (Graph-RAG). A diferencia de los sistemas basados en bases de datos vectoriales planas, estructura la información mediante nodos (entidades) y aristas (relaciones semánticas). Este enfoque permite al LLM no solo recuperar datos locales, sino navegar a través de rutas conectadas, posibilitando una comprensión global del dominio y un razonamiento estructurado mucho más robusto.

4.2.1. Categorías según las estructuras de grafos

- **Basado en conocimiento:** Transforma texto no estructurado en Grafos de Conocimiento explícitos, modelando nodos y aristas.
- **Basado en índices:** Mantiene el texto original y emplea el grafo para conectar semánticamente fragmentos de texto.
- **Híbrido:** Combina las fortalezas de los grafos de conocimiento y de índices para abordar tareas de razonamiento de alta complejidad.

El funcionamiento de Graph-RAG comparte la estructura de tres etapas del RAG tradicional, pero adaptadas a una arquitectura de grafos. Primero, se estructura la información externa mediante grafos, ya sea creando representaciones explícitas del conocimiento o estableciendo redes de indexación semántica. A continuación, se emplean planificadores basados en grafos que no solo evalúan la similitud semántica, sino también la coherencia lógica, extrayendo subgrafos altamente relevantes para la consulta. Finalmente, el modelo fusiona el subgrafo recuperado con la consulta inicial del usuario.

4.2.2. Beneficios ante sistemas basados en bases de datos vectoriales

- **Representación avanzada del conocimiento:** Al capturar relaciones de múltiples saltos (multi-hop), el sistema comprende contextos jerárquicos y resuelve con mayor precisión consultas ambiguas o que requieren deducciones complejas.
- **Flexibilidad de integración:** Permite unificar en una sola estructura topológica diversas fuentes de información, abarcando datos estructurados, semiestructurados y texto no estructurado.
- **Eficiencia computacional y escalabilidad:** El recorrido de las bases de datos de grafos reduce la longitud del contexto necesario. Estudios evidencian que Graph-RAG genera respuestas utilizando entre un 26% y un 97% menos de tokens. Además, permite actualizar la base de conocimiento en tiempo real añadiendo nodos, sin necesidad de reindexar el corpus completo.
- **Interpretabilidad y trazabilidad:** La estructura de grafos hace que el proceso de razonamiento sea transparente y auditable. Es posible visualizar el camino lógico que el sistema ha seguido para generar una respuesta, una característica indispensable en sectores críticos como el legal, el financiero o el sanitario.

4.3. RAG Tradicional vs. Graph-RAG

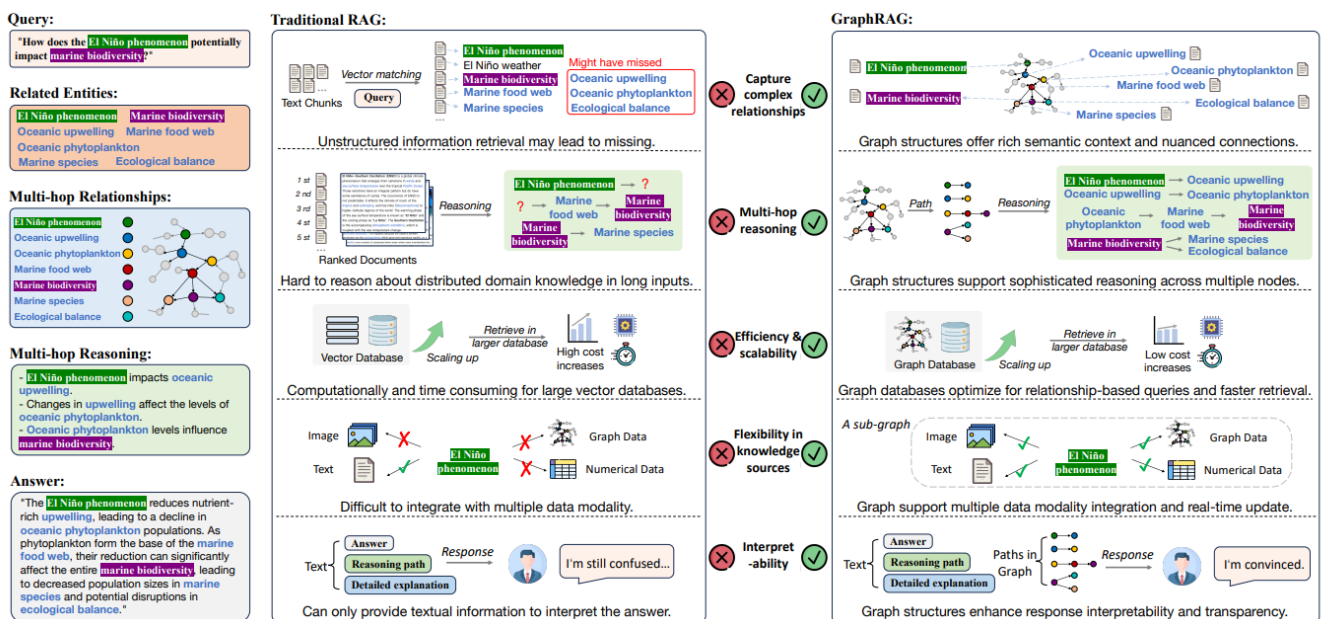


Figura 3 Comparación entre RAG Y Graph-RAG

La transición hacia el paradigma Graph-RAG surge para superar las limitaciones estructurales de la indexación vectorial pura. Mientras que el modelo RAG organiza el corpus en chunks sin conexión lógica, Graph-RAG emplea bases de datos orientadas a grafos. Esta arquitectura modela relaciones y jerarquías explícitas que preservan la integridad semántica global mediante enlaces topológicos. Con ello, se habilita un razonamiento de múltiples pasos (multi-hop) capaz de conectar conceptos dispersos para resolver consultas complejas. En consecuencia, Graph-RAG optimiza radicalmente la eficiencia al reducir el consumo de tokens entre un 26% y un 97%, permite actualizaciones dinámicas sin necesidad de reindexar el corpus completo, y transforma la opacidad del filtrado vectorial en un proceso de inferencia auditable, trazable y visualizable.

4.4. FCA

4.4.1. Conceptos previos

El Análisis Formal de Conceptos fue introducido por Rudolf Wille en la década de 1980 basándose en la teoría de retículos de Birkhoff.

Por tanto, el objetivo es rectificar y clarificar algunas premisas de la teoría de retículos, sobre lo que el propio Wille afirmaba:

“El objetivo y el significado del FCA como teoría matemática sobre conceptos y sus jerarquías es apoyar la comunicación racional entre seres humanos mediante el desarrollo matemático de estructuras conceptuales apropiadas que se puedan manipular con la lógica”

4.4.2. Deficiones

1. Una **relación binaria** R se define como

$$R = \{ (a_1, a_2) : (a_1, a_2) \in A_1 \times A_2 \wedge R(a_1, a_2) = \text{Verdadero} \}$$

Siendo un conjunto formado por pares que están relacionados entre sí, mediante alguna propiedad determinada. Si se desea representar gráficamente la definición anterior, sería posible mediante un retículo, término que proviene de la forma de los diagramas de Hasse. Estos son los siguientes conceptos a introducir

2. Un **retículo** es una estructura algebraica con dos operaciones binarias y que forma un conjunto parcialmente ordenado, que cumple la propiedad de que toda pareja $a, b \in A$ tiene un único supremo $\sup(a, b) \in A$ y un único ínfimo $\inf(a, b) \in A$.

3. Un **diagrama de Hasse** es una representación gráfica de un conjunto parcialmente ordenado finito. Además de las definiciones anteriores, es interesante presentar la operación de derivación, pues nos servirá para trabajar con la relación P entre objetos y atributos.

Dados dos subconjuntos $X \subseteq O$ y $Y \subseteq A$, tenemos que:

$$X' = \{y \in A: \forall x \in X, P(x, y)\}$$

$$Y' = \{x \in O: \forall y \in Y, P(x, y)\}$$

Es decir, X' recoge los atributos comunes a todos los objetos en X e Y' los objetos que tienen todos los atributos en Y .

4. Un **contexto formal** $K := (G, M, I)$ está formado por dos conjuntos G y M y una relación I entre estos dos. Los elementos de G se denominan objetos y los de M atributos del contexto. Para expresar que un objeto g se encuentra en una relación I con un atributo m , se escribe gIm y se lee 'el objeto g posee el atributo m '
5. Un **concepto formal** del contexto (G, M, I) es una tupla (A, B) con $A \subseteq G$, $B \subseteq M$, $A' = B$ y $B' = A$. Se denomina a A como **extensión** y a B **intensión**. $B(G, M, I)$ constituye el conjunto de todos los conceptos de un contexto (G, M, I) .

La operación de **extensión** permite, dado un conjunto de atributos, computar qué objetos los poseen, mientras que la **intensión** nos dice, dado una serie de objetos, qué atributos poseen. Además, también es posible el cómputo de implicaciones, que constituyen una serie de reglas que se cumplen para distintos conjuntos de atributos.

6. Se dice que dos subconjuntos A y B proporcionan la **implicación** $A \rightarrow B$ si se cumple $A' \subseteq B'$, es decir, que A y B constituyen todos los objetos que tienen en común sus atributos, respectivamente

La solidez demostrada por las estructuras del FCA contrasta con la naturaleza probabilística de los LLMs. Mientras que los LLMs destacan en la comprensión del lenguaje y la generación de texto, carecen de mecanismos internos de verificación lógica. Esta complementariedad fundamenta el paradigma de la **IA Neuro-Simbólica**, el cual propone integrar el motor de razonamiento matemático del FCA como una capa de autoridad lógica sobre la recuperación de información estructurada (Graph-RAG). De este modo, las deficiencias de alucinación de las redes neuronales se restringen y verifican en tiempo real mediante las implicaciones deterministas extraídas de los datos.

4.4.3. Ejemplo sencillo con el paquete fcaR

Con el objetivo de asentar todos estos conceptos teóricos, resulta conveniente plantear un ejemplo práctico que permita clarificar las ideas más complejas. Para ello, se emplea el paquete fcaR, el cual proporciona una implementación sencilla e intuitiva de los métodos clásicos usados en FCA, así como de los algoritmos desarrollados en las investigaciones de los tutores de este TFG.

El primer paso consiste en definir el conjunto de datos sobre el que se desarrollará el ejemplo. En este caso, el conjunto de objetos está compuesto por una selección de animales. Por su parte, el conjunto de atributos representa diversas características biológicas o de hábitat que estos poseen, tales como si son mamíferos, terrestres, acuáticos o si tienen la capacidad de volar.

	Mamífero	Ave	Terrestre	Acuático	Vuela
Gato	X		X		
Perro	X		X		
Delfín	X			X	
Ballena	X			X	
Gorrión		X	X		X
Pingüino		X		X	

Primero construimos el contexto formal mediante:

```
> fc <- FormalContext$new(datos_animales)
```

Esto nos proporciona la estructura básica a partir de la que trabajar. Para empezar, es posible computar los conceptos e implicaciones a través de los siguientes métodos, respectivamente:

```
> fc$find_concepts()
> print(fc$concepts)
A set of 10 concepts:
1: ({Gato, Perro, Delfín, Ballena, Gorrión, Pingüino}, {})
2: ({Delfín, Ballena, Pingüino}, {Acuatico})
3: ({Gato, Perro, Gorrión}, {Terrestre})
4: ({Gorrión, Pingüino}, {Ave})
5: ({Pingüino}, {Ave, Acuatico})
6: ({Gorrión}, {Ave, Terrestre, Vuela})
7: ({Gato, Perro, Delfín, Ballena}, {Mamifero})
```

```
8: ({Delfín, Ballena}, {Mamifero, Acuatico})
9: ({Gato, Perro}, {Mamifero, Terrestre})
10: ({}, {Mamifero, Ave, Terrestre, Acuatico, Vuela})
```

```
> fc$find_implications()
> print(fc$implications)
Implication set with 4 implications.
Rule 1: {Vuela} -> {Ave, Terrestre}
Rule 2: {Terrestre, Acuatico} -> {Mamifero, Ave, Vuela}
Rule 3: {Ave, Terrestre} -> {Vuela}
Rule 4: {Mamifero, Ave} -> {Terrestre, Acuatico, Vuela}
```

Existen dos funciones que permiten realizar operaciones de derivación: *intent()* y *extent()*. Estas operaciones sirven para obtener el conjunto de atributos que comparten una serie de objetos y viceversa.

Por ejemplo, dado el conjunto *Gato, Perro*, obtendríamos lo siguiente:

```
> S_objetos <- Set$new(attributes = fc$objects)
> S_objetos$assign(Gato = 1, Perro = 1)
> print(fc$intent(S_objetos))
{Mamifero, Terrestre}
```

Por otro lado, si se desea conocer qué animal es a su vez *mamífero y acuático*, obtendríamos los siguientes resultados:

```
> S_atributos <- Set$new(attributes = fc$attributes)
> S_atributos$assign(Mamifero = 1, Acuatico = 1)
> print(fc$extent(S_atributos))
{Delfín, Ballena}
```

5

FCA en Graph-RAG

5.1. Síntesis Neuro-Simbólica

La integración del Análisis Formal de Conceptos (FCA) en las arquitecturas Graph-RAG conforma un *sistema de IA neuro-simbólica* basado en la Teoría del Proceso Dual. Este paradigma logra una sinergia al combinar dos enfoques complementarios:

- **El Sistema 1 (LLMs):** Aporta rapidez, reconocimiento de patrones y comprensión del lenguaje natural. En este entorno, el LLM deja de ser un depositario de hechos para actuar exclusivamente como un analizador semántico que traduce las consultas a un contexto formal.
- **El Sistema 2 (FCA):** Compensa la naturaleza probabilística del LLM aportando un razonamiento analítico y determinista. Extrae jerarquías y reglas absolutas (bases de implicaciones de Duquenne-Guigues) a partir de los datos, libres de ambigüedad.

Este flujo de trabajo permite que el motor de FCA actúe como la autoridad lógica y mecanismo de salvaguarda del sistema. Al escrutar las inferencias propuestas por el LLM contra restricciones matemáticas estrictas, el sistema detecta y bloquea cualquier alucinación o contradicción estocástica, garantizando respuestas completamente fiables.

5.2. LLM como analizador semántico y traductor formal

Para que el motor de FCA pueda ejercer su función, la naturaleza operativa del LLM dentro de la arquitectura experimenta un cambio fundamental. El LLM ya no se utiliza como un depositario estático de hechos o como una memoria paramétrica de la cual extraer conocimiento, sino que asume el rol de una interfaz de alto nivel y un analizador semántico avanzado.

La tarea principal de la red neuronal en esta fase consiste en llevar a cabo un proceso de **autoformalización**. El LLM actúa como un traductor que procesa el grafo de conocimiento y el texto original en lenguaje natural para estructurarlos y convertirlos en un Contexto Formal matemático. De este modo, el LLM es el encargado de desglosar semánticamente el dominio para identificar qué entidades actúan como objetos (el conjunto matemático G), qué características o propiedades operan como atributos (el conjunto M), y delimitar las conexiones que conforman la relación de incidencia (I).

Además, el LLM ejerce de puente bidireccional entre el usuario y la capa matemática. Por un lado, cuando el sistema recibe una consulta en lenguaje natural, el modelo la analiza y la traduce hacia el vocabulario simbólico del retículo de conceptos. Por otro lado, una vez que el motor simbólico ha navegado por el retículo y ha devuelto las reglas y conceptos válidos, el LLM retoma su capacidad generativa para sintetizar una respuesta fluida y coherente, sabiendo que la generación del lenguaje está estrictamente restringida y anclada a la ruta deductiva que el FCA le ha proporcionado.

5.3. FCA como autoridad lógica

Este control se ejerce mediante la base de implicaciones de Duquenne-Guigues, la cual representa las verdades inmutables del dominio. El FCA impone esta autoridad en dos fases clave para bloquear las alucinaciones:

1. **Anclaje previo a la generación:** Antes de que el LLM responda, el sistema localiza el concepto formal exacto que corresponde a la consulta. Restringiéndose el espacio de búsqueda del modelo a un entorno lógicamente completo y consistente, reduciendo la invención de datos.
2. **Verificación posterior:** Durante la inferencia, cada afirmación propuesta por el LLM se descompone y se contrasta contra la base de implicaciones. Si el modelo probabilístico genera una conclusión que viola estas reglas matemáticas, el sistema detecta la contradicción y bloquea o corrige la alucinación.

5.4. Explicabilidad y trazabilidad

La integración del Análisis Formal de Conceptos transforma este paradigma al proporcionar una explicabilidad fundamentada en la intensión de los conceptos. Dado que el retículo de conceptos constituye una estructura matemática determinista y exhaustiva, permite registrar una ruta lógica reproducible para cada inferencia.

De este modo, el retículo garantiza que cada respuesta del sistema sea auditable paso a paso. Un usuario o auditor puede examinar la traza exacta del proceso de decisión:

- La consulta inicial se mapea a un concepto formal específico dentro del retículo.
- Se expande la intensión del nodo para revelar todos los atributos y propiedades asociadas.
- Se aplican las reglas de la base de implicaciones matemáticas para validar la deducción.
- La conclusión se fundamenta en la evidencia documental exacta (la extensión del concepto).

5.5. Caso ilustrativo

Supóngase un sistema Graph-RAG encargado de procesar manuales técnicos de componentes electrónicos. En la fase inicial, el Modelo de Lenguaje extrae la información y genera una serie de trípticas relacionales para construir el grafo de conocimiento:

- [Sensor A] tiene propiedad [Impermeable]
- [Sensor B] tiene propiedad [Sumergible]
- [Sensor A] utiliza [Protocolo I2C]
- [Sensor B] utiliza [Protocolo I2C]

En una arquitectura Graph-RAG estándar, el grafo resultante constituye una red de relaciones plana y asociativa. Si un usuario introduce la consulta "**¿El Sensor B es resistente al agua (impermeable)?**", el sistema buscará la conexión directa. Al no existir una arista explícita que conecte [Sensor B] con [Impermeable], el LLM generativo se enfrenta a un vacío de información, lo que eleva el riesgo de producir una alucinación o de emitir una respuesta negativa incorrecta por falta de contexto.

Es en esta limitación donde interviene la capa lógica del FCA. Al transformar este subgrafo en un contexto formal, el motor matemático calcula el Retículo de Conceptos y descubre automáticamente la jerarquía subyacente. Como resultado, extrae una implicación lógica fundamental del dataset: el atributo [*Sumergible*] implica matemáticamente el atributo [*Impermeable*] (*Sumergible* $\rightarrow$ *Impermeable*).

Esta regla deductiva representa una verdad absoluta dentro del dominio de los datos que el grafo, por sí solo, no era capaz de garantizar ni inferir de manera determinista. En la fase final de generación, el sistema inyecta esta restricción lógica validada en el prompt del Modelo de Lenguaje. Provisto de esta autoridad lógica, el LLM es capaz de inferir con total certidumbre que, dado que el Sensor B es sumergible, hereda obligatoriamente la propiedad de ser impermeable, formulando así una respuesta correcta, fundamentada y libre de alucinaciones.

6

Desarrollo aplicación Shiny

7

Validación y pruebas

8

Conclusiones y líneas futuras

Referencias

- [1] I. H. Witten and E. Frank, Data Mining: Practical Machine Learning Tools and Techniques, 2nd ed. San Francisco, CA: Morgan Kaufmann, 2005.
(Libro)
-

Apéndice A.

Manual de Usuario



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga